CMSC 430 Project 4 Documentation

Author: Joe Merrill

Course: CMSC 430 Compiler Theory and Design

Project: Project 4

Date: April 13, 2026

> This report accompanies the cleaned source archive for Project 4. The full regression run of the supplied tests is saved in `Project 4 Output.txt`, and the full run of the custom tests created for this report is saved in `Project 4 Custom Output.txt`.

Discussion of Approach

I approached Project 4 by treating it as a semantic-analysis extension of the earlier compiler work. The supplied skeleton already handled the basic parse structure, so I first mapped each rubric item to the file where it belonged: token-level updates in `scanner.l`, grammar and semantic actions in `parser.y`, symbol lookups in `symbols.h`, and type checking helpers in `types.cc` and `types.h`. That let me work feature by feature without losing track of where each semantic rule should be enforced.

The first group of changes focused on the type system. I added support for real types, hexadecimal integer literals, and arithmetic coercion from integer to real inside arithmetic expressions. From there, I expanded the semantic checks so the compiler could detect mismatched variable initializations, invalid function returns, invalid list declarations and subscripts, incorrect use of arithmetic and remainder operators, invalid comparisons between characters and numeric expressions, and type mismatches across `when`, `switch`, `if`, and `fold` constructs.

The second group of changes focused on symbol-table behavior. I kept separate symbol tables for scalars and lists so the compiler could continue to distinguish the two forms of identifiers, then added duplicate-name checks before insertion. That made it possible to detect duplicate scalar declarations, duplicate list declarations, and collisions between names already used in the other table. Parameter declarations reuse the same scalar table, so duplicate parameter and local-variable names are caught the same way.

I verified the implementation in two layers. First, I rebuilt the compiler from source using Flex, Bison, and g++. Second, I ran the supplied regression tests that cover the rubric items from the project handout. After the official suite matched the expected outputs, I added custom tests of

my own to cover name-collision edge cases and valid semantic behavior that was not shown directly in the provided handout output. That extra pass gave me more confidence that the semantic checks were working for both required and self-created cases.

Test Plan

Supplied Regression Suite

The provided Project 4 test set was used as the primary regression suite. Together these tests cover the real-type additions, arithmetic coercion, symbol-table errors, type-checking errors, and the multiple-error case required by the rubric.

File	Aspect Tested	Expected Outcome
`valid1.txt`	Accepts real variables and real return types.	Compiled successfully.
`valid2.txt`	Treats hexadecimal literals as integers.	Compiled successfully.
`valid3.txt`	Performs integer-to-real coercion in arithmetic expressions.	Compiled successfully.
`valid4.txt`	Allows widening from integer to real in assignments and returns.	Compiled successfully.
`semantic1.txt`	Type mismatch on variable initialization.	One semantic error.
`semantic2.txt`	`when` branch type mismatch.	One semantic error.
`semantic3.txt`	Non-integer `switch` expression.	One semantic error.
`semantic4.txt`	Mismatched `case` result types.	One semantic error.
`semantic5.txt`	Arithmetic operator used with a character value.	One semantic error.
`semantic6.txt`	Undeclared scalar identifier.	One semantic error.
`semantic7.txt`	Undeclared list identifier.	One semantic error.
`semantic8.txt`	List elements with mixed types.	One semantic error.
`semantic9.txt`	Declared list type differs from element type.	One semantic error.
`semantic10.txt`	List subscript is not an integer expression.	One semantic error.
`semantic11.txt`	Character literal compared to numeric expression.	One semantic error.

`semantic12.txt`	Exponentiation and unary negation used with characters.	Two semantic errors.
`semantic13.txt`	Remainder operator used with a real operand.	One semantic error.
`semantic14.txt`	`if`, `elsif`, and `else` branches do not match in type.	One semantic error.
`semantic15.txt`	`fold` applied to a non-numeric list.	One semantic error.
`semantic16.txt`	Narrowing during variable initialization.	One semantic error.
`semantic17.txt`	Narrowing function return.	One semantic error.
`semantic18.txt`	Duplicate scalar and duplicate list declarations.	Two semantic errors.
`semantic19.txt`	Multiple semantic errors in one program.	Seven semantic errors.
`semantic20.txt`	Function return type mismatch.	One semantic error.

Custom Tests Created for This Report

To strengthen the test plan beyond the supplied handout cases, I created four additional programs. These custom tests target name-collision edge cases and valid semantic behavior that complements the official suite.

File	Aspect Tested	Expected Outcome
`custom1_parameter_duplicate.txt`	Detects a duplicate scalar when a local variable reuses a parameter name.	One semantic error: duplicate scalar.
`custom2_scalar_list_collision.txt`	Detects a duplicate identifier when a list reuses a scalar name.	One semantic error: duplicate list.
`custom3_valid_character_compare.txt`	Confirms that character-to-character relational comparisons are legal.	Compiled successfully.
`custom4_valid_real_fold.txt`	Confirms that `fold` accepts a numeric list variable of type real.	Compiled successfully.

The four custom test source files are stored in `Project 4 Test Data` beside the supplied suite.

Representative Compiler Runs

The screenshots below show the compiler being run on both supplied and custom tests. The custom tests are included to demonstrate self-created verification beyond the instructor-provided

regression suite.

```
valid3.txt

PS> Get-Content .\Project 4 Test Data\valid3.txt | .\Project 4 Skeleton Code\compile.exe


   1  -- Program with a Real Variable
   2
   3  function main returns real;
   4      a: real is 4 + 4.5;
   5  begin
   6      a;
   7  end;
   8

Compiled Successfully
```

Figure 1. Supplied regression case `valid3.txt`, demonstrating integer-to-real coercion in arithmetic expressions.

```
semantic19.txt

PS> Get-Content .\Project 4 Test Data\semantic19.txt | .\Project 4 Skeleton Code\compile.exe


   1  // Multiple Semantic Errors
   2
   3  function main returns integer;
   4      value: integer is 4.5;
Semantic Error, Illegal Narrowing Variable Initialization
   5      numbers: list of real is (1, 2, 3);
Semantic Error, List Type Does Not Match Element Types
   6      one: integer is '1';
Semantic Error, Type Mismatch on Variable Initialization
   7  begin
   8      if value > 0 then
   9          fold left + ('a', 'b') endfold;
Semantic Error, Fold Requires A Numeric List
  10      elsif name = 'N' then
Semantic Error, Undeclared Scalar name
  11          fold right * (1, 2.5) endfold;
Semantic Error, List Element Types Do Not Match
  12      else
  13          when value < 10, 1 : 1.5;
Semantic Error, When Types Mismatch
  14      endif;
  15  end;
  16

Lexical Errors 0
Syntax Errors 0
Semantic Errors 7
```

Figure 2. Supplied regression case `semantic19.txt`, demonstrating recovery and reporting of multiple semantic errors in one compilation.

```
custom1_parameter_duplicate.txt

PS> Get-Content .\Project 4 Test Data\custom1_parameter_duplicate.txt | .\Project 4 Skeleton Code\compile.exe


   1  -- Duplicate Parameter and Local Variable
   2
   3  function main value: integer returns integer;
   4      value: integer is 1;
Semantic Error, Duplicate Scalar value
   5  begin
   6      value;
   7  end;
   8

Lexical Errors 0
Syntax Errors 0
Semantic Errors 1
```

Figure 3. Custom test `custom1_parameter_duplicate.txt`, showing that a parameter name cannot be reused by a local scalar declaration.

```
custom2_scalar_list_collision.txt

PS> Get-Content .\Project 4 Test Data\custom2_scalar_list_collision.txt | .\Project 4 Skeleton Code\compile.exe


   1  -- Scalar and List Share One Identifier
   2
   3  function main returns integer;
   4      item: integer is 5;
   5      item: list of integer is (1, 2, 3);
Semantic Error, Duplicate List item
   6  begin
   7      item;
   8  end;
   9

Lexical Errors 0
Syntax Errors 0
Semantic Errors 1
```

Figure 4. Custom test `custom2_scalar_list_collision.txt`, showing that a list identifier cannot reuse an existing scalar name.

```
custom3_valid_character_compare.txt

PS> Get-Content .\Project 4 Test Data\custom3_valid_character_compare.txt | .\Project 4 Skeleton Code\compile.exe


   1  -- Valid Character Comparison
   2
   3  function main returns integer;
   4  begin
   5      if 'A' < 'Z' then
   6          1;
   7      else
   8          0;
   9      endif;
  10  end;
  11

Compiled Successfully
```

Figure 5. Custom test `custom3_valid_character_compare.txt`, showing a valid character comparison that compiles successfully.

custom4_valid_real_fold.txt

```
PS> Get-Content .\Project 4 Test Data\custom4_valid_real_fold.txt | .\Project 4 Skeleton Code\compile.exe


   1  -- Valid Fold Over Real List Variable
   2
   3  function main returns real;
   4      values: list of real is (1.5, 2.0, 3.5);
   5  begin
   6      fold left + values endfold;
   7  end;
   8

Compiled Successfully
```

Figure 6. Custom test `custom4_valid_real_fold.txt`, showing a valid fold over a real list variable.

Lessons Learned

The most important lesson from Project 4 is that semantic analysis is where the language rules start to feel real. The parser can accept a program that is syntactically well formed, but semantic analysis decides whether that program actually makes sense. Working through the project made the separation between syntax and meaning much clearer to me, especially in cases like valid versus invalid type coercion or valid versus invalid use of identifiers.

I also learned how useful it is to centralize semantic checks in helper functions instead of scattering complex logic throughout the grammar. Moving the semantic rules into functions such as `checkAssignment`, `checkArithmetic`, `checkRelational`, and the list-related helpers made the parser actions easier to read and made the behavior easier to verify with targeted tests. That organization reduced the chance of inconsistent error handling between similar grammar productions.

Another important lesson was that symbol-table testing needs to include collisions that are easy to overlook. The supplied tests covered duplicate scalars and duplicate lists, but adding custom tests around parameter-name reuse and scalar-list collisions helped confirm that the compiler rejects duplicates consistently across declaration contexts. That kind of targeted edge-case testing is especially valuable in compiler projects because a small oversight in scope or lookup logic can create confusing downstream errors.

Improvements That Could Be Made

- Add an automated regression script that runs every supplied and custom test case and compares the compiler output against expected results.
- Refactor the semantic helper functions into more clearly grouped categories, such as assignment checks, control-flow checks, arithmetic checks, and list checks.
- Extend the symbol-table layer to support nested scopes explicitly, which would make the design easier to reuse in later compiler phases.

- Add a packaging script that rebuilds the compiler, refreshes test outputs, and recreates the final submission archive in one command.

Submission Contents

The final Project 4 submission package produced for this repository includes the following deliverables:

- `CMSC 430 Project 4- Joe Merrill.zip`: cleaned source archive containing only the required `.l`, `.y`, `.cc`, `.h`, and `makefile` files.
- `CMSC 430 Project 4-Joe Merrill.pdf`: exported documentation report.
- `Project 4 Custom Output.txt`: full console output for the custom tests created for this report.